

ARCHITECTURE HANDOVER DOCUMENT

DQTool

Enterprise architecture brief for a data-quality, profiling, anomaly-detection and rule-execution application.

Windows-first

Python 3.11+

NiceGUI

SQLite

Optional local AI

Purpose

Define, run and monitor data-quality checks across CSV and enterprise data sources.

Current architecture

One deployable Python process containing the browser UI, application services and scheduler.

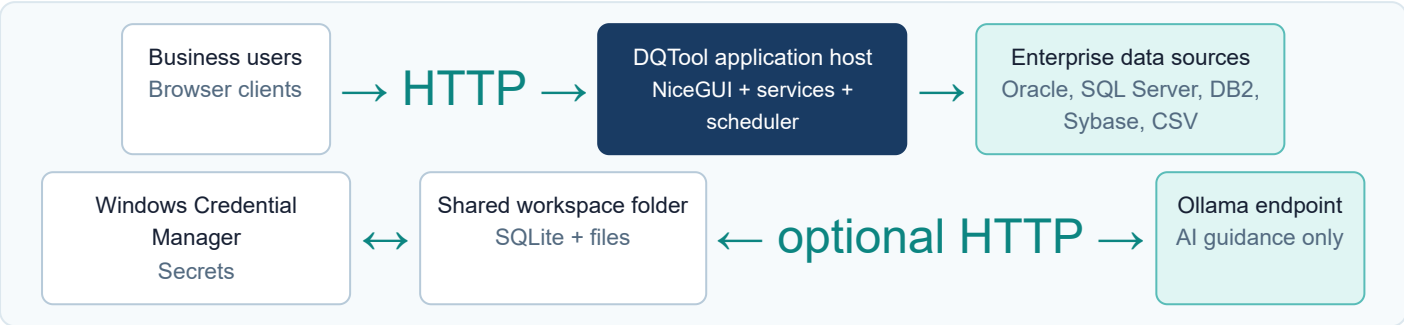
Key design question

Determine the controls and services required to evolve from team deployment to enterprise operation.

Prepared from the DQTool source and technical documentation. This describes the implemented architecture, followed by enterprise decisions to validate.

1. System context

DQTool is a browser-accessed application for data stewards, project administrators and users. It reads from configured CSV and database sources, stores workspace/project metadata locally, and can optionally obtain AI guidance from an Ollama endpoint.



Network default: the server listens on 127.0.0.1:8080. LAN access requires explicitly setting DQTOOL_HOST=0.0.0.0 and allowing inbound Windows Firewall traffic.

2. Application containers and responsibilities

Presentation NiceGUI Web UI Browser session, login, dashboards, forms, preview, export and permission-aware actions.	Workspace & project Workspace / Storage services Accounts, memberships, roles, projects, SQLite migrations and CRUD.	Source access Connector service Database/CSV connection tests, target discovery, previews and source queries.
Execution Execution service Rule evaluation, status/counts/runtime, bounded failed-row evidence and result persistence.	Analysis Profiling service Column statistics, drift, anomaly/content findings, GDPR review flags and rule ideas.	Automation & AI Scheduler / Ollama service Minute polling for due rules; optional AI explanations and recommendations.

3. Data stores and locations

<code><workspace-root>/</code>	
<code> dqtool_workspace.sqlite</code>	shared accounts, projects, access
<code><project-folder>/</code>	
<code> dqtool_project.sqlite</code>	rules, connections, profiles, runs, schedules
<code> uploads/</code>	browser-uploaded CSV source files
<code> results/</code>	failed-row evidence files
<code> exports/</code>	generated CSV/XLSX reports

Store	Data	Architectural concern
Workspace SQLite	Users, salted password hashes, projects and memberships.	Back up with all projects; assess SQLite concurrency and shared-filesystem suitability.

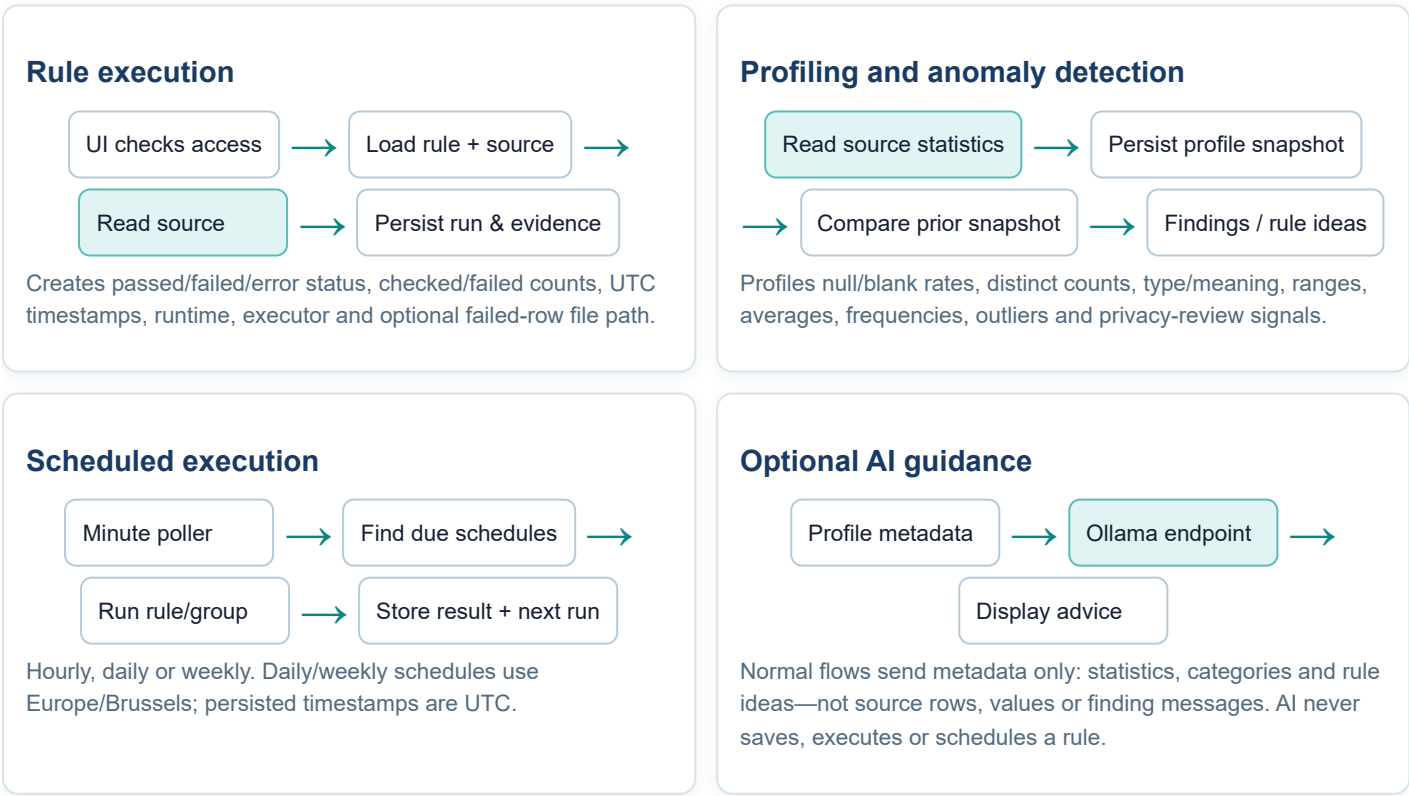
Project SQLite	Connection definitions without passwords, rules/groups, schedules, run history and profiles.	Each project is independently portable but needs consistent backup with project files.
Uploads, results, exports	CSV source files, failed-row evidence, generated reports.	Potentially sensitive data: apply ACL, encryption, retention, DLP and malware-scanning policy.
Windows Credential Manager	Per-user source passwords and optional Cloudflare AI service token.	Local to the Windows host/user; central-server process identity is important.
%LOCALAPPDATA%\DQTool	Local settings, workspace selection, AI endpoint/model, session signing secret.	Host-local configuration; needs deployment/configuration management approach.

4. Integrations and technology

Integration	Technology	Use	Direction
Browser client	HTTP, NiceGUI cookies/sessions	User access to DQTool.	Inbound
CSV	Filesystem and DuckDB	Uploaded source files, profile, preview and rule evaluation.	Read; uploads write to project folder
Oracle	oracledb	Profile, preview and rule execution.	Outbound read access recommended
SQL Server / DB2 / Sybase ASE / Sybase IQ	pyodbc + native ODBC driver	Profile, preview and rule execution.	Outbound read access recommended
Ollama (optional)	HTTP; default <code>http://localhost:11434, qwen3:8b</code>	AI explanations and recommended next actions.	Outbound
Cloudflare Access (optional)	HTTP service-token headers	Access to protected remote Ollama endpoint.	Outbound

Native ODBC drivers are operating-system dependencies on the DQTool host. The currently supported defaults are SQL Server ODBC Driver 17, IBM DB2 ODBC Driver, Adaptive Server Enterprise and Sybase IQ (all overrideable per connection).

5. Primary data flows



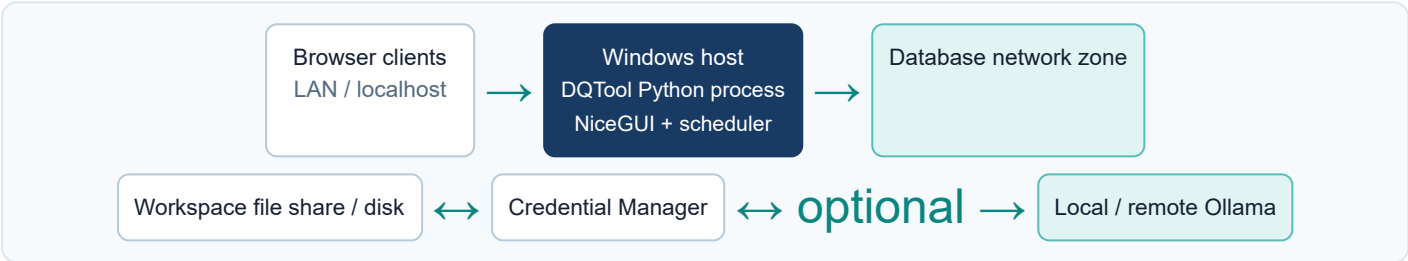
6. Identity, access and secrets



- **Workspace Admin:** creates accounts/projects, assigns access and administers all projects.
 - **Project Admin:** administers members and all items in a project.
 - **Project User:** accesses permitted project work and own items.
 - Items may be private, shared project-wide or shared with named users.
- Application passwords are salted PBKDF2-SHA256 hashes in workspace SQLite.
 - Database passwords are not in project SQLite; they are stored per Windows user in Credential Manager.
 - Cloudflare AI credentials also use Credential Manager.
 - Source accounts should be read-only; custom SQL is trusted-user functionality.

Trust-boundary focus: uploaded files, failed-row evidence and exports may contain raw business or personal data. The architecture should explicitly show filesystem access controls and the source-data boundary. Any opt-in AI sample-value flow should be separately approved.

7. Deployment model and constraints



- One application process hosts UI, execution, profiling and scheduling.
- There is no message broker, work queue, worker tier, API gateway, load balancer or separate scheduler service.
- Schedules only run while the application process is alive. Run as a managed Windows service or equivalent for unattended operation.
- Horizontal scaling is not currently suitable: SQLite, local credentials and in-process scheduling create node affinity and duplicate-execution risk.
- SQLite must be assessed for concurrent writes, locking on any network share, backup consistency and high-availability needs.

8. Enterprise architecture decisions

Decision area	Question to resolve
Operating model	Is DQTool a local tool, departmental central server or enterprise service?
Identity	Should local accounts be replaced with OIDC, SAML, Active Directory or another corporate identity provider?
Secrets	Should Windows Credential Manager be replaced or supplemented with an enterprise secrets vault?
Persistence	Should SQLite and filesystem storage move to managed database and object/document storage for concurrency, audit and HA?
Scheduling	Should the in-process poller move to a durable enterprise scheduler/job runner?
AI governance	May metadata leave the application boundary? Which approved model endpoint and data-classification controls apply?
Data protection	What retention, encryption, DLP, ACL, malware scanning and disposal policies govern uploaded CSVs, evidence and exports?
Custom SQL	Which roles may use it, which database identities, timeouts, allowlists and audit controls are required?

